



← Back to student guides

 Docker

Containers

3 levels

133 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

24

sections

32

examples





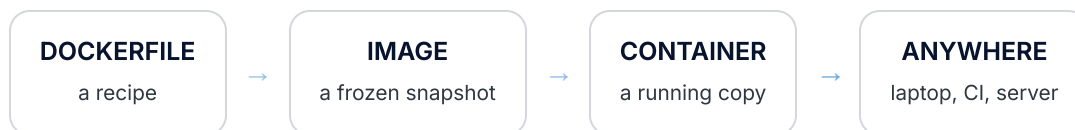
two services together, and hand a colleague one command that starts your whole project.

Mid-level and Senior take the same topics further; nothing here is thrown away.

Each section ends with a **Try it** task. Do them as you go. They take a few minutes each, and these concepts only stick once you have watched your own container start, crash, and start again properly.

What Docker is, and the problem it solves

Docker packages an application **together with the environment it needs to run** (the language runtime, the libraries, the system packages, the configuration) into a single file called an image. You can then run that image as a container on any machine that has Docker, and it behaves the same way.



The diagram is the whole idea. Compare it to what came before.

Before containers, getting a project running meant a page of setup instructions: install this version of Python, install these system libraries, set these environment variables, start Postgres, create a database. Every developer executed those steps slightly differently, so every developer's machine was slightly different. Then the server was different again (a different distribution, a different libc, an older OpenSSL) and code that passed every test locally failed in production for reasons nobody could reproduce.

"It works on my machine" describes a real, structural problem rather than a careless developer: the environment was never part of the thing you tested. Docker makes it part of the thing you ship.

Two consequences of that design explain most of what follows. Notice them now rather than discovering them later.



breaks, `git log` tells you who changed the environment and why. This sounds administrative; in practice it is the single biggest reason teams stop being afraid of their deployment.

A container is disposable. You do not repair a container, you delete it and start a new one from the image. That is what makes containers safe to experiment with, and it is also the biggest source of confusion for newcomers, because anything you wrote inside the container disappears with it. We will come back to that repeatedly.

What people use it for:



One-command development

A new team member clones the repository, runs one command, and has the app plus its database running.



Honest CI

Tests run in the same image that will run in production, so a pass means something.



Predictable deploys

The artifact that passed CI is byte-for-byte the artifact that ships. No rebuild, no drift.



Disposable tools

Run Postgres, Redis, or a specific Python version for ten minutes and leave nothing installed behind.

You need little to follow along: Docker installed, a terminal, and any small project. A brand-new folder with a three-line web app is the best place to experiment, because a failed build costs you nothing.

► TRY IT

1. Write down, from memory, every step needed to run one of your projects on a brand-new laptop.
2. Mark each step as **environment** (install a runtime, a library, a service) or **your code**.
3. Count the environment steps.



Reason this guide is worth your afternoon.

Container versus virtual machine

This is the most common interview opener and the most common source of wrong mental models, so get it straight before anything else.

✓ Container

- Shares the host's kernel
- Starts in milliseconds
- Tens to hundreds of megabytes
- Isolation from kernel features (namespaces, cgroups)
- Run dozens on a laptop

✗ Virtual machine

- Ships a whole guest kernel and OS
- Starts in tens of seconds
- Gigabytes
- Isolation from a hypervisor: stronger
- Run a handful on a laptop

Keep this sentence: **a container is a process on your machine with a restricted view of the filesystem, the network, and the process table.** It is not a small computer. There is no second operating system booting inside it, which is why it starts in the time it takes to start any other program.

That restricted view is convincing enough to be confusing. Inside a container, `ls /` shows a different filesystem, `ps aux` shows only the container's own processes, and `hostname` returns something unfamiliar. None of that is emulation. The Linux kernel is showing that process a different picture. Senior level covers how, because the security consequences matter. For now, the practical takeaways are:

Because a container...	You get
Shares the host kernel	Startup in milliseconds, tiny memory overhead
Has no guest OS	Images measured in megabytes, not gigabytes



is just a process	<code>docker stats</code> looks like a process monitor, because it is
Uses kernel-level isolation	Weaker separation than a VM: real, but not a security boundary for untrusted code

Docker on macOS and Windows does use a VM

Containers are a Linux kernel feature, so on a Mac or a Windows machine Docker Desktop runs a small Linux virtual machine and your containers live inside it. This is why file access to mounted host folders is slower there than on Linux, and it explains a class of platform-specific surprises covered in the Tips & Tricks section.

► TRY IT

1. Run `docker run --rm -it alpine sh`, then inside it run `ls /`, `ps aux`, and `hostname`.
2. In a second terminal on your host, run `ps aux | grep sh` and find that shell.
3. Type `exit` in the container.

Expected result – inside, `ps aux` shows one or two processes and the hostname is a random hex string. On the host, that same shell is visible as an ordinary process. That is the whole model in one experiment: one process, two views.

Image, container, registry: the three nouns

Three words that get used interchangeably in conversation and mean different things in practice.

	Image	Container	Registry
Is	A read-only template	A running instance of an image	A server that stores images



Analogy	A class - an installer	An object - the installed program	A package index
Count	One	Many, from the same image	Docker Hub, GHCR, ECR
Changes	Immutable	Thin writable layer, lost on removal	Versioned by tag and digest
Command	<code>docker build</code> , <code>docker pull</code>	<code>docker run</code>	<code>docker push</code> , <code>docker pull</code>

BASH

Copy

```
docker run hello-world # pulls the image from a registry, then runs a container
```

That one command exercises all three ideas: Docker looks for the `hello-world` image locally, does not find it, pulls it from Docker Hub, creates a container from it, runs it, and the container exits when its program finishes.

One image, many containers

`docker run -d nginx` three times gives you three independent containers from one image. They share the image's read-only layers on disk, so the third costs almost no extra space, and each gets its own thin writable layer for anything it changes.

► TRY IT

1. Run `docker run hello-world` and read the output it prints. It describes exactly the steps above.
2. Run it a second time and notice that nothing is downloaded, because the image is now local.
3. Run `docker images` to see the image, and `docker ps -a` to see both containers you created.

Expected result – one image and two containers. Two containers from one run command each, both stopped, both still taking up a little disk until you remove



Check your setup: client, daemon, registry

Before writing anything, confirm what you have. Three commands tell you everything that matters.

BASH

Copy

```
docker version      # client and server versions – the server line is the daemon
docker info         # storage driver, root directory, resources, warnings
docker run hello-world # end-to-end proof: pull, create, run
```

`docker version` printing a **Client** section but failing on **Server** is the single most common setup problem, and it has one meaning: the Docker daemon is not running. Start Docker Desktop, or on Linux `sudo systemctl start docker`.

Know the shape of what you just installed, because it explains several error messages:

YOUR MACHINE

docker CLI

A thin client. Sends every request over a socket. It builds nothing itself



DOCKERD: THE DAEMON DOES THE WORK

Builder

Receives the build context, runs each instruction, writes layers

Runtime



Image layers

Read-only, shared between containers

Container

One process, plus a thin writable layer

Registry

Docker Hub, GHCR, ECR: pulled on demand

Why this matters: the daemon must be running for any command to work; the build context is uploaded **to** the daemon, which is why its size affects build speed; and on Linux the socket is root-owned, which is why commands need **sudo** until your user joins the **docker** group.

The **docker** command you type is only a client. It sends your request over a socket to a background service, the **daemon**, which builds images, starts containers, and manages storage. This matters in three practical ways: the daemon needs to be running for any command to work, files are sent **to** the daemon at build time (which is why build context size affects speed), and on Linux the socket is root-owned, which is why Docker commands need **sudo** until your user is added to the **docker** group.

Symptom	Means
Cannot connect to the Docker daemon	The daemon is not running
permission denied ... /var/run/docker.sock	Your user is not in the docker group
no space left on device	Docker's storage area is full: see the cleanup section

► TRY IT



3. On Linux, run `ls -l /var/run/docker.sock` and note who owns it.

Expected result – a Server section (so the daemon is alive), and an `info` output that tells you where images are stored and how much memory containers can use. Knowing the root directory is what makes "my disk is full" solvable later.

Your first containers

Now run something real. These six commands are the core loop of working with Docker, and you will type them thousands of times.

BASH

Copy

```
# 1. Run a web server in the background, mapped to localhost:8080
docker run -d -p 8080:80 --name web nginx

# 2. Confirm it is running
docker ps

# 3. Read what it is printing
docker logs web

# 4. Get a shell inside it and look around
docker exec -it web sh

# 5. Stop it
docker stop web

# 6. Delete it
docker rm web
```

Open <http://localhost:8080> after the first command and nginx is serving a page, with nothing installed on your machine, no configuration file edited, and nothing to uninstall afterwards.



Part	Does
<code>docker run</code>	Create a container from an image and start it
<code>-d</code>	Detached: run in the background and give you your prompt back
<code>-p 8080:80</code>	Publish host port 8080 to container port 80
<code>--name web</code>	Name it, instead of accepting a random one like <code>nostalgic_bardeen</code>
<code>nginx</code>	The image. No tag means <code>nginx:latest</code>

Without `-d` the container runs in the foreground and your terminal shows its output until you press Ctrl+C. That is often what you want while developing, and always what you want the first time you run something new. You get to see it fail.

Use `--rm` for anything throwaway

`docker run --rm -it python:3.11 python` gives you a Python 3.11 REPL and leaves nothing behind. Without `--rm`, every experiment leaves a stopped container behind until `docker ps -a` becomes a wall of text and your disk fills.

► TRY IT

1. Run all six commands above in order, visiting `localhost:8080` between steps 1 and 5.
2. Inside the container at step 4, run `ls /usr/share/nginx/html` and `cat /etc/nginx/nginx.conf`.
3. Now run `docker run --rm -it python:3.11 python`, print `import sys; sys.version`, and exit.
4. Run `docker ps -a` and confirm the Python container is not listed.

Expected result – a working web server you never installed, a Python version you do not have on your machine, and, thanks to `--rm`, no leftovers from the second experiment. That contrast between the nginx container you had to remove and the Python one that cleaned itself up is the habit to take away.



There are hundreds of Docker subcommands. In day-to-day work you use about fifteen.

Command	Does
<code>docker run IMAGE</code>	Create and start a container
<code>docker ps</code> / <code>docker ps -a</code>	Running containers / all containers including stopped
<code>docker logs -f NAME</code>	Stream a container's output
<code>docker exec -it NAME sh</code>	Open a shell inside a running container
<code>docker stop</code> / <code>start</code> / <code>restart NAME</code>	Stop, start, restart a container
<code>docker rm NAME</code>	Delete a stopped container
<code>docker images</code>	List local images
<code>docker rmi IMAGE</code>	Delete an image
<code>docker build -t name:tag .</code>	Build an image from a Dockerfile
<code>docker pull</code> / <code>push IMAGE</code>	Download from / upload to a registry
<code>docker inspect NAME</code>	Full JSON: config, mounts, network, state
<code>docker stats</code>	Live CPU and memory per container
<code>docker cp NAME:/path ./</code>	Copy a file out of (or into) a container
<code>docker system df</code>	How much disk Docker is using
<code>docker system prune</code>	Reclaim space from unused objects

The `docker run` flags worth memorising early:



<code>-d</code>	Detached. run in the background
<code>-it</code>	Interactive + TTY, so you get a usable shell
<code>-p 8080:80</code>	Publish host port 8080 → container port 80
<code>--name web</code>	Give it a name instead of a random one
<code>--rm</code>	Delete the container automatically when it exits
<code>-e KEY=value</code>	Set an environment variable
<code>--env-file .env</code>	Set many environment variables from a file
<code>-v name:/path</code>	Mount a volume or a host folder
<code>-w /app</code>	Set the working directory
<code>--entrypoint sh</code>	Replace the image's entrypoint: the debugging escape hatch
<code>--restart unless-stopped</code>	Restart it automatically after a crash or reboot

Two of those deserve a note now because they save real time. `docker exec -it NAME sh` only works on a **running** container; if the container has already exited there is nothing to exec into, and you want `--entrypoint sh` instead. `docker inspect` is the answer to almost every "but I set that" argument. It shows the container's configuration after every default, image setting, and command-line override has been resolved.

► TRY IT

1. Start a container: `docker run -d --name web -p 8080:80 nginx .`
2. Run `docker inspect web` and find three things in the JSON: the image it came from, the published ports, and its environment variables.
3. Now get the same answers as one line each: `docker inspect web --format '{{.Config.Image}}'` and `docker inspect web --format '{{json .NetworkSettings.Ports}}'` .
4. Run `docker stats --no-stream` and note the memory the container is using.

Expected result — a wall of JSON, then the same facts as single lines. `--format` is the difference between `inspect` being unusable and being the first tool you



The container lifecycle, state by state

A container moves through a small number of states, and most beginner confusion comes from not knowing which one it is in.



created `docker create`

The writable layer exists and configuration is fixed; nothing is running yet.



running `docker start` / `docker run`

The main process is alive. `docker ps` shows it.



paused `docker pause`

Processes frozen in place. Rare, but it exists.



exited **the process ended, or** `docker stop`

SIGTERM, then SIGKILL after ten seconds. The filesystem and logs still exist.



gone `docker rm`

The writable layer is deleted. Anything not in a volume is lost forever.

The rule that explains most surprises: **a container lives as long as its main process**. Not as long as you want it to, and not until you stop it. It lives as long as process number one inside it keeps running.

BASH

Copy

```
docker run ubuntu          # exits immediately
docker run -it ubuntu bash  # stays, because bash has a terminal to read from
docker run -d nginx         # stays, because nginx is a server that does not exit
docker run --rm alpine echo hi # prints "hi", exits, removes itself
```



read, so it finishes immediately, and when the main process finishes, the container is done. Adding `-it` attaches a terminal and gives the shell something to wait for. A container is not a machine you log into; it is one process with a restricted view.

An **exited** container still exists. Its filesystem, its logs, and its exit code are all still there, which is what you need in order to debug it, and it is why `docker ps -a` fills up with corpses if you never use `--rm`.

► TRY IT

1. Run `docker run --name ghost ubuntu`. Confirm it is not in `docker ps` but is in `docker ps -a`.
2. Read its exit code from the STATUS column, then run `docker logs ghost`.
3. Run `docker start ghost`. The same container starts again and exits again.
4. Now run `docker run --name alive -it ubuntu bash`, type `sleep 30`, and from a second terminal run `docker stop alive` and watch what happens in the first.

Expected result – the first container exits instantly with code 0. It succeeded, it had nothing to do. The second survives because `-it` gave the shell a terminal, and `docker stop` visibly ends it. Those two runs together explain about a third of all "my container won't stay up" questions.

Images, tags, and registries

An image reference has three parts, and Docker fills in the ones you leave out, which is where surprises come from.

TEXT

Copy

ghcr.io/my-org/myapp:1.4.2

ghcr.io	my-org	myapp	1.4.2
registry	owner	name	tag



<code>nginx</code>	<code>docker.io/library/nginx:latest</code>	Official image, latest tag
<code>nginx:1.27</code>	<code>docker.io/library/nginx:1.27</code>	Pinned to a minor version
<code>myuser/myapp</code>	<code>docker.io/myuser/myapp:latest</code>	A user's image on Docker Hub
<code>ghcr.io/org/app:1.4.2</code>	Exactly that	A different registry

BASH

Copy

```
docker pull nginx:1.27          # download without running
docker images                   # what you have locally
docker tag myapp:1.0 myuser/myapp:1.0
echo "$TOKEN" | docker login ghcr.io -u USERNAME --password-stdin
docker push myuser/myapp:1.0
docker rmi nginx:1.27           # delete a local image
```

:latest does not mean "the newest version"

It is just the tag Docker assumes when you do not give one, and it points at whatever the publisher last pushed under that name. It can move under you between two builds an hour apart. Pin a real tag in every `FROM` line and in every deployment (`nginx:1.27` , not `nginx`) so that "what is running?" always has an answer. Mid-level shows why even a version tag is not fully immutable, and what is.

Choosing a base image is your first real decision, and the sensible default is narrower than people expect:

Base	Size	Use it when
<code>python:3.11</code>	~1 GB	You need compilers and headers; fine for experiments
<code>python:3.11-slim</code>	~150 MB	The sensible default for most applications
<code>python:3.11-alpine</code>	~50 MB	Size matters and you have tested it



ubuntu:22.04

~78 MB

you want a general-purpose OS and will install everything yourself

Start with `-slim` , not Alpine

Alpine looks like the obvious choice because it is smallest, but it uses `musl` instead of `glibc`, which means many prebuilt Python wheels and Node native modules do not apply and get compiled from source instead. The build gets slower, needs a toolchain, and often ends up `larger` than the slim variant. Reach for Alpine deliberately and with a measurement, not by default.

► TRY IT

1. Pull three variants: `docker pull python:3.11` , then `python:3.11-slim` , then `python:3.11-alpine` .
2. Run `docker images python` and read the SIZE column.
3. Run `docker run --rm python:3.11-slim python -c "print('works')"` and then the same on the alpine tag.
4. Try `docker run --rm python:3.11-alpine bash` and note the error.

Expected result – roughly 1 GB, 150 MB, and 50 MB for the same Python version, and the Alpine image has no `bash` at all, only `sh` . That error is the one you will hit again the first time you try to `exec` into a slim image.

Writing a Dockerfile

Running other people's images is useful. Building your own is the point.

A `Dockerfile` is a plain text file, named exactly that, describing how to construct an image. Each instruction takes the result of the previous one and adds something.



```
FROM python:3.11-slim          # the base image to start from

WORKDIR /app                   # cd into /app, creating it if needed

COPY requirements.txt .        # copy one file from your machine into the image
RUN pip install --no-cache-dir -r requirements.txt  # runs at BUILD time

COPY . .                       # now copy the rest of the source

EXPOSE 8000                    # documentation: this app listens on 8000

CMD ["python", "-m", "uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

BASH

Copy

```
docker build -t myapp:1.0 .    # the "." is the build context, not the Dockerfile
docker run -d -p 8000:8000 myapp:1.0
```

Let me walk through every line, because this small file contains the entire conceptual model.

`FROM python:3.11-slim` says "start from this image". You are never building from nothing; you are adding to something. This must be the first instruction.

`WORKDIR /app` sets the working directory for every instruction after it, and for the container at run time. It creates the directory if it does not exist, which is why you rarely need `mkdir`.

`COPY requirements.txt .` copies from the **build context** (your project folder) into the image at the current `WORKDIR`. Note it copies **into the image**, permanently. This is a build-time action, and the file is baked in.

`RUN pip install ...` executes a command **while building** and saves the result as part of the image. The installed packages are now in the image forever. This is the instruction people misuse most, and the next section explains why.

`COPY . .` copies everything else. It comes after the install deliberately. That ordering is the single biggest speed lever available to you, and it has its own section below.

`EXPOSE 8000` is documentation. It publishes nothing at all.



The `-t myapp:1.0` in the build command is the tag you are giving the result, in `name:tag` form. Skip it and you get an untagged image identified only by a hash, which is how "dangling" images accumulate.

The two mistakes everyone makes here

RUN happens at build time; CMD happens at run time. Putting your application's start command in a **RUN** makes the build hang forever, waiting for a server that never exits.

EXPOSE does not publish anything. It is a note for humans and tooling. Without `-p 8000:8000` on `docker run`, nothing on your machine can reach the container.

► TRY IT

1. Create a folder with three files: a one-line `requirements.txt` containing `fastapi[standard]`, an `app.py` with a single route, and the Dockerfile above.
2. Run `docker build -t myapp:1.0 .` and read the output. Every line is one instruction, and Docker tells you what it is doing.
3. Run it: `docker run -d -p 8000:8000 --name myapp myapp:1.0`, then visit `localhost:8000`.
4. Now break it deliberately: change `CMD` to `RUN` on the last line and rebuild. Cancel it with Ctrl+C after ten seconds.

Expected result – a working image you built, and then a build that hangs forever because you asked Docker to run a web server as a build step. Causing that hang on purpose is worth the sixty seconds. It makes the build-time versus run-time distinction permanent.

The Dockerfile instructions worth knowing

There are about eighteen instructions. These are the ones that appear in real Dockerfiles.



FROM	-	Sets the base image. Always first
WORKDIR	Build + run	Sets the working directory for what follows
COPY src dst	Build	Copies from the build context into the image
ADD	Build	Like COPY , but also unpacks archives and fetches URLs
RUN	Build	Executes a command and saves the result as a layer
ENV KEY=value	Build + run	Sets an environment variable in the image
ARG KEY=value	Build only	A variable you can pass with --build-arg
EXPOSE	-	Documents a port. -p publishes
USER	Run	Switches the user for what follows
VOLUME	Run	Declares a path as a mount point
CMD	Run	Default command, overridable by docker run img args
ENTRYPOINT	Run	The fixed executable; CMD becomes its arguments
HEALTHCHECK	Run	How Docker decides whether the container is healthy
LABEL	-	Metadata: maintainer, source repository, version

Three of them are worth extra words now, because they are commonly confused.

COPY versus ADD . They look interchangeable. **ADD** additionally auto-extracts local tar archives and can download a URL, behaviour that is convenient once and surprising thereafter. Use **COPY** always, and reach for **ADD** only when you specifically want tar extraction.

ENV versus ARG . **ARG** exists only during the build; **ENV** persists into the running container. Both are visible to anyone who can pull the image, so neither is a place for a secret.



```
ENV LOG_LEVEL=info          # a default, present at run time and overridable with -e
ENV PYTHONUNBUFFERED=1      # makes Python flush stdout, so docker logs works properly
```

`PYTHONUNBUFFERED=1` deserves a special mention. Python buffers stdout when it is not a terminal, so in a container your log lines can sit in a buffer for a long time and `docker logs` looks empty while the app is working. One `ENV` line removes an entire class of confusion.

`LABEL` costs nothing and answers "where did this image come from?" six months later:

DOCKERFILE

Copy

```
LABEL org.opencontainers.image.source="https://github.com/my-org/myapp"
LABEL org.opencontainers.image.description="Checkout API"
```

► TRY IT

1. Add `ENV PYTHONUNBUFFERED=1` and the two `LABEL` lines to your Dockerfile, and rebuild.
2. Add `ARG APP_VERSION=dev` above `FROM`, then `ENV APP_VERSION=${APP_VERSION}` after it. Rebuild with `--build-arg APP_VERSION=1.0`.
3. Confirm the value arrived: `docker run --rm myapp:1.0 env | grep APP_VERSION`.
4. Override it at run time: `docker run --rm -e APP_VERSION=hotfix myapp:1.0 env | grep APP_VERSION`.

Expected result – `1.0` from the build argument, then `hotfix` from the run-time flag. You have just seen the two-stage configuration model that the whole "build once, run anywhere" idea depends on: bake a default, override per environment.

Layers: why `COPY` order decides your build time

This is the single biggest speed lever a beginner can pull, and it takes one minute to apply.



writable layer on top of the stack.



Docker caches layers. When you rebuild, it walks the instructions from the top and reuses each layer whose inputs have not changed. **The moment one layer is invalidated, every layer after it must be rebuilt.**

Now the consequence. Your source code changes constantly; your dependency list barely ever changes. So the order of two instructions decides whether every build reinstalls all your dependencies:

✓ **Fast: dependencies first**

- `COPY requirements.txt .`
- `RUN pip install ...`
- `COPY. .`
- Editing source reuses the cached install
- Rebuild: a couple of seconds

✗ **Slow: everything first**

- `COPY. .`
- `RUN pip install ...`
- Any source edit invalidates the install layer
- Every build reinstalls everything
- Rebuild: a couple of minutes

The same reasoning applies inside a single `RUN`. Package manager metadata and cleanup must happen in the `same` instruction, because a later instruction cannot remove anything from an earlier layer:

DOCKERFILE

Copy

```
# Wrong: the stale package index gets cached, and the lists stay in the image
RUN apt-get update
RUN apt-get install -y curl

# Right: one layer, cleaned up inside that same layer
RUN apt-get update \
  && apt-get install -y --no-install-recommends curl \
  && rm -rf /var/lib/apt/lists/*
```



by anyone who can pull it. This is why `COPY .env .` followed by `RUN rm .env` does not remove the secret; it just hides it from a casual look. Mid and Senior levels return to this, because it is the mechanism behind most leaked credentials in images.

You can see the layers you built:

BASH

Copy

```
docker history myapp:1.0          # every layer, its size, and the instruction
docker image inspect myapp:1.0 --format '{{.Size}}'
```

► TRY IT

1. Build your image, then change one character in `app.py` and build again. Time it.
2. Now swap the order: put `COPY. .`

Expected result – before

the `pip install` line. Build, edit one character, build again. Time it.

3. Put the ordering back, then run `docker history myapp:1.0` and find the biggest layer.
4. Watch the build output for the words `CACHED`. That is Docker telling you which layers it skipped.

Expected result – a rebuild that takes a second or two in one ordering and a full reinstall in the other. Seeing `CACHED` disappear from your build output the moment you move one line is the clearest possible demonstration of why layer order is a design decision.

The build context and `.dockerignore`

When you run `docker build -t myapp .`, that trailing dot sets the **build context** rather than naming the Dockerfile: it is the directory that gets packaged up and sent to the Docker



`.git` , `node_modules` , a virtualenv, build output, and datasets all get transferred. Second, and more seriously, **anything in the context can end up in the image**, including the `.env` file you never meant to ship.

`.dockerignore` fixes both. It sits next to your Dockerfile and uses the same syntax as `.gitignore` .

`.dockerignore`

Copy

```
# Version control and tooling
.git
.gitignore
.github

# Dependencies – reinstalled inside the image
node_modules
__pycache__
*.pyc
.venv
venv

# Local configuration and secrets
.env
.env.*
*.pem
*.key

# Build output and noise
dist
build
*.log
coverage
.pytest_cache

# Editor and OS files
.vscode
.idea
.DS_Store
```



Confirm what landed in the image

`docker run --rm myapp:1.0 ls -la /app` lists what is there. This one command settles both "is my file missing because of `.dockerignore`?" and "did I accidentally ship `.env`?", and it is much faster than reasoning about it.

► TRY IT

1. Note the "transferring context" size in your build output.
2. Create a large junk file in your project: `dd if=/dev/zero of=big.bin bs=1M count=200` (or any 200 MB file). Rebuild and compare the context size.
3. Add `big.bin` to `.dockerignore` and rebuild.
4. Run `docker run --rm myapp:1.0 ls -la /app` and confirm `big.bin` is not inside.

Expected result – the context jumps by 200 MB and the build visibly slows, then returns to normal once the file is ignored. That "transferring context" line is worth glancing at on every build. It is an early warning that something large has crept into your project.

CMD versus ENTRYPOINT

Both describe what runs when the container starts, and the difference between them is a favourite interview question because it reveals whether you have shipped an image.

	CMD	ENTRYPOINT
ls	The default command	The fixed executable
Arguments after <code>docker run img</code>	Replace it entirely	Are appended to it



USE FOR	A default that users may want to change	A container that always runs one program
---------	---	--

DOCKERFILE

Copy

```
# CMD only: the whole command is a default
CMD ["uvicorn", "app:app", "--host", "0.0.0.0"]
# docker run myapp          → runs uvicorn
# docker run myapp python -V → runs python -V instead

# ENTRYPOINT + CMD: fixed program, default arguments
ENTRYPOINT ["uvicorn", "app:app"]
CMD ["--host", "0.0.0.0", "--port", "8000"]
# docker run myapp          → uvicorn app:app --host 0.0.0.0 --port 8000
# docker run myapp --port 9000 → uvicorn app:app --port 9000
```

The pairing in the second example is the pattern worth copying for a real service: [ENTRYPOINT](#) names the program, [CMD](#) supplies arguments a user might reasonably want to override.

There is a second, less obvious distinction that matters more than the first one: **exec form versus shell form**.

DOCKERFILE

Copy

```
CMD ["node", "server.js"]    # exec form (a JSON array): your process is PID 1
CMD node server.js          # shell form: PID 1 is /bin/sh, which runs your process
```

Always use the exec form (a JSON array)

With the shell form, process 1 inside the container is `/bin/sh`, and it does **not** pass signals on to your application. So `docker stop` sends `SIGTERM`, your app never sees it, and ten seconds later Docker kills the container hard, mid-request, mid-transaction. Note the syntax detail too: the array uses **double quotes**, because it is JSON. Single quotes are a build error.



Two rules cover almost every port problem you will hit:

Inside a container, your app must listen on `0.0.0.0` . Not `127.0.0.1` , and not `localhost` . Binding to loopback inside a container means "reachable only from inside this container", so `-p` appears to do nothing at all. Most frameworks default to loopback, which is why nearly every containerised app command has a `--host 0.0.0.0` in it.

One host port can only be used once. `bind: address already in use` means something else, often a previous container you forgot to remove, already has it. `docker ps` then `docker rm -f` clears it.

The most common "port mapping doesn't work" cause

Your app is listening on `127.0.0.1` inside the container. Confirm it from inside: `docker exec web sh -c 'netstat -tlnp 2>/dev/null || ss -tlnp'` . If the address column shows `127.0.0.1:8000` rather than `0.0.0.0:8000` , the mapping was never the problem.

► TRY IT

1. Run your app bound to loopback on purpose: change the command to `--host 127.0.0.1` and run with `-p 8000:8000` . Visit the page and watch it fail.
2. From inside, check what is listening: `docker exec NAME sh -c 'ss -tlnp || netstat -tlnp'` .
3. Change it back to `0.0.0.0` and confirm it works.
4. Now try to start a second container on the same host port and read the error.

Expected result – a refused connection that looks exactly like a broken port mapping, and an `ss` output showing `127.0.0.1:8000` as the reason. Then a clear "address already in use" for the duplicate. Those are two of the three port errors you will ever see.

Configuration: environment variables and files



BASH

Copy

```
docker run -e LOG_LEVEL=debug -e DATABASE_URL="$DB_URL" myapp:1.0
docker run --env-file ./local.env myapp:1.0
docker run --rm myapp:1.0 env # see what the container actually got
```

local.env

Copy

```
LOG_LEVEL=debug
DATABASE_URL=postgres://postgres:secret@db:5432/app
FEATURE_NEW_CHECKOUT=true
```

The mechanisms and where each one belongs:

Mechanism	Set at	Visible in the image	Use for
<code>ENV</code> in the Dockerfile	Build	Yes	Sensible non-secret defaults
<code>ARG</code> + <code>--build-arg</code>	Build only	Yes, in <code>docker history</code>	Base versions, build flags
<code>-e KEY=value</code>	Run	No	Per-environment configuration
<code>--env-file</code>	Run	No	Many values at once, locally
A mounted file	Run	No	Credentials

Neither `ENV` nor `ARG` is a secret

Both are recorded in image metadata and readable with `docker history` by anyone who can pull the image. A password passed as a build argument is in that history forever, and deleting the file in a later layer does not help. At this level the rule is simple: **secrets**



Precedence, most specific first: `-e` on the command line beats `--env-file` , which beats `ENV` in the Dockerfile. That layering is what makes one image work everywhere.

► TRY IT

1. Add `ENV LOG_LEVEL=info` to your Dockerfile and rebuild.
2. Run `docker run --rm myapp:1.0 env | grep LOG_LEVEL` to see the baked default.
3. Run it again with `-e LOG_LEVEL=debug` and confirm the override wins.
4. Write a `local.env` with three variables, run with `--env-file local.env` , and confirm all three arrived.
5. Now check whether a build argument is invisible: add `ARG DEMO_TOKEN=abc` , rebuild, and run `docker history --no-trunc myapp:1.0 | grep DEMO_TOKEN` .

Expected result – the override behaves as expected, and the last step finds your build argument sitting in the image history in plain text. Seeing that yourself is far more persuasive than being told "don't put secrets in `ARG` ".

Logs, and getting inside a container

Two commands cover almost all day-to-day investigation.

BASH

Copy

```
docker logs web                # everything the container has printed
docker logs -f --tail 50 web    # follow, starting from the last 50 lines
docker logs --since 10m web     # only the last ten minutes
docker logs -t web              # with timestamps

docker exec -it web sh          # a shell inside a RUNNING container
docker exec web env             # run one command without a shell session
docker exec -u root -it web sh  # as root, when you need to install a debug tool
```



inside the container instead, `docker logs` shows nothing and your logs are with the container.

Log to stdout, always

It is what `docker logs` reads, what every log shipper collects, and what every orchestrator expects. Writing to a file inside a container hides your logs, fills the writable layer, and gains you nothing. Most frameworks do this by default; if yours writes to a file, change the configuration rather than working around it.

If `bash` is not found, use `sh`

Slim and Alpine images frequently have no bash. `docker exec -it NAME sh` works nearly everywhere. If an image has no shell at all, a distroless or scratch image, you cannot exec into it, and that is deliberate rather than broken.

Three more commands complete the toolkit:

BASH

Copy

```
docker inspect web          # resolved config: entrypoint, env, mounts, network, st
docker stats                # live CPU and memory, per container
docker cp web:/etc/nginx/nginx.conf ./    # copy a file out to look at it properly
```

► TRY IT

1. Start your app and run `docker logs -f NAME` in one terminal while you hit the endpoint from another.
2. Get a shell inside it and explore: `ls -la /app`, `env`, `ps aux`, `cat` a config file.
3. Change your app to write a log line to a file instead of stdout, rebuild, and confirm `docker logs` is now empty.
4. Copy that file out with `docker cp` to prove the lines were written.



secret logging, and you now have it first hand.

Keeping data: volumes

Delete a container and its writable layer goes with it. Anything you want to survive must live in a **volume**.

BASH

Copy

Named volume – Docker manages the storage. Right for databases and app state.

```
docker run -d --name db \  
  -e POSTGRES_PASSWORD=secret \  
  -v pgdata:/var/lib/postgresql/data \  
  postgres:16
```

Bind mount – a real folder on your machine. Right for live-reload development.

```
docker run -d -p 8000:8000 -v "$(pwd)":/app myapp:dev
```

Read-only mount, for configuration the container must not change

```
docker run -d -v ./config.yaml:/etc/app/config.yaml:ro myapp:1.0
```

```
docker volume ls
```

```
docker volume inspect pgdata
```

```
docker volume rm pgdata          # only works if no container is using it
```

✓ Named volume

- `-v pgdata:/var/lib/postgresql/data`
- Docker owns the storage location
- Portable across machines and hosts
- Ownership initialised correctly for you
- Right for databases and app state

✗ Bind mount

- `-v "$(pwd)":/app`
- A real folder on your host
- Host-path dependent, so not portable
- Permission-sensitive: keeps the host's ownership
- Right for development, wrong for production data



outcomes. Named volumes survive `docker rm`, move with your project, and get sensible permissions. Bind mounts are for when you [want](#) the host's actual files, chiefly so your editor and the container see the same source code.

A bind mount hides whatever was already there

Mounting your project folder over `/app` replaces the image's `/app` entirely, including anything installed there during the build. For Node projects this is the notorious `node_modules` disappearance. The fix is an anonymous volume over the subdirectory, which the Tips section shows.

► TRY IT: THE ONE THAT MAKES IT STICK

1. Run Postgres with **no** volume, create a table with `docker exec`, then `docker rm -f` it and start it again. Look for your table.
2. Now run it with `-v pgdata:/var/lib/postgresql/data`, create the table again, remove the container, and start a new one with the same volume.
3. Run `docker volume ls` and `docker volume inspect pgdata`.
4. Finally, bind-mount your source into your app container and edit a file on your host while it is running.

Expected result – the table is gone in the first case and present in the second, from a different container. Destroying your own data once, deliberately, is the fastest way to never do it accidentally.

Networking: how containers find each other

One container is rarely enough. As soon as you have an app and a database, they need to talk, and the way they find each other surprises everyone once.



```
docker run -d --name db --network appnet -e POSTGRES_PASSWORD=secret postgres:10
docker run -d --name api --network appnet -p 8000:8000 \
-e DATABASE_URL=postgres://postgres:secret@db:5432/postgres myapp:1.0
```

The connection string says `db`, not `localhost`. On a network you created, Docker runs a DNS server and every container is reachable by its name. That is the whole mechanism.

Network	Behaviour
<code>bridge</code> (the default)	Containers get IPs, but no name resolution
A network you create	Same, plus automatic DNS by container name
<code>host</code>	No network isolation at all; the container uses the host's stack
<code>none</code>	No networking

The important row is the first two. If you never create a network, your containers land on the default bridge, where names do not resolve and you are left using IP addresses. Creating a network is one command and it is why Compose, which does it for you, feels so much easier.

Inside a container, `localhost` means `that container`

It is the single most common networking mistake. A container cannot reach a sibling on `localhost`, and it cannot reach a service running on your host machine that way either. Use the container name for a sibling; use `host.docker.internal` to reach your host from inside a container on Docker Desktop.

► TRY IT

1. Start two containers with **no** network flag, then from one run `docker exec -it NAME sh` and try `ping other`. It fails to resolve.
2. Now `docker network create appnet` and start both with `--network appnet`. Try `ping other` again.
3. From inside the app container, try `wget -q0- http://localhost:5432` and then `wget -q0- http://db:5432`.
4. Run `docker network inspect appnet` and find both containers listed.



means the app container itself. That is the single most valuable networking fact at this level.

Do not run as root

By default the process inside a container runs as root. Adding a user is two lines and it is the highest-value security change available to a beginner.

Dockerfile

Copy

```
FROM python:3.11-slim

# Create a real user with a fixed numeric id
RUN useradd --create-home --uid 10001 appuser

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Give the app's files to that user as they are copied
COPY --chown=appuser:appuser . .

USER 10001                                # everything after this runs unprivileged

EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Two details in there are deliberate. `USER` comes **after** the installs, because `pip install` into system directories needs the privileges you are about to drop. The id is numeric rather than a name, because that form is unambiguous to tooling and to orchestrators.

Root in a container is root on the host kernel

There is no hypervisor between them. If a process escapes its container (through a kernel bug, a careless mount, or an over-permissive flag) it lands on the host with



covers the full picture.

Order matters, and getting it wrong produces permission errors that look mysterious:

Do	Why
Install dependencies before <code>USER</code>	Installing to system paths needs privilege
<code>COPY --chown=...</code>	Otherwise files are root-owned and your user cannot write them
Use a numeric uid in <code>USER</code>	Unambiguous, and survives image inspection tooling
Listen on a port above 1024	Binding below 1024 requires privilege

► TRY IT

1. Run `docker run --rm myapp:1.0 id` before adding a user. Note `uid=0(root)` .
2. Add the three lines above, rebuild, and run `id` again.
3. Try to write somewhere privileged from inside: `docker run --rm myapp:1.0 sh -c 'touch /etc/test'` .
4. Now put `USER 10001`

Expected result – before
the `pip install` line and rebuild, to see the failure that ordering causes.

Expected result – root first, then `uid=10001` , then a clean "permission denied" proving the restriction is real, then a failed build proving why `USER` goes last. Four short runs and you understand the whole pattern.

Docker Compose: many containers, one file

By now a single `docker run` line has six flags on it, and you have two containers plus a network to start in the right order. That is what Compose is for: it moves everything you were



compose.yaml

Copy

```
services:
  api:
    build: . # build from the Dockerfile here
    ports: ['8000:8000']
    environment:
      DATABASE_URL: postgres://postgres:secret@db:5432/app
      LOG_LEVEL: debug
    depends_on: [db]
    restart: unless-stopped

  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: app
    volumes: ['pgdata:/var/lib/postgresql/data']

volumes:
  pgdata:
```

BASH

Copy

```
docker compose up -d # build if needed, create the network, start everything
docker compose ps    # what is running, and its health
docker compose logs -f api # follow one service
docker compose exec api sh # a shell in a service
docker compose down   # stop and remove containers and the network
docker compose down -v # ...and delete the named volumes too
```

Every key in that file maps to something you already know:

Compose key	The <code>docker run</code> equivalent
<code>build: .</code>	<code>docker build -t</code>
<code>image:</code>	The image argument



ports:	-p
environment: / env_file:	-e / --env-file
volumes:	-v
restart:	--restart
depends_on:	Nothing: Compose adds startup ordering
The service name	--name , and the DNS hostname

Two things Compose gives you for free are worth calling out. It **creates a network** and puts every service on it, so `api` reaches the database at the hostname `db` with no configuration at all. It makes the whole stack one unit: `up` , `down` , `logs` , and `ps` operate on all of it.

`depends_on` waits for `started` , not `ready`

It waits for the database container to start, not for Postgres inside it to accept connections, and those are several seconds apart. So your app's first query fails on a cold start, intermittently, in a way that looks like a bug in your code. Mid-level fixes this properly with health checks; for now, know that the gap exists and that retrying the connection in your app is the right instinct.

Use Compose even for a single service

A `compose.yaml` documents the ports, environment, and volumes that would otherwise live only in your shell history. "How do I run this?" becomes `docker compose up` , which is also the answer you want in your README.

► TRY IT

1. Write the `compose.yaml` above for your app and run `docker compose up` in the foreground so you can read both services' logs interleaved.
2. Stop it with Ctrl+C, then start it detached with `-d` and use `docker compose ps` and `docker compose logs -f api` .



Expected result – two services started by one command, reachable from each other by name, and a volume that survives `down` but not `down -v`. That last distinction is worth burning in: `-v` is how people delete their local database by accident.

Debugging a container that will not start

This is the skill everything else depends on, and it has an order. Work down the list rather than guessing.

01 Read the logs

`docker logs NAME`. An exited container keeps its output until you remove it, so this works even after a crash. Most failures explain themselves here and you can stop at step one.

02 Check the exit code

`docker ps -a` shows it in the STATUS column.

0

= finished normally ·

1

= application error ·

125

= bad `docker run` flags ·

126

= command not executable ·

127

= command not found ·

137

= killed, usually out of memory ·

143

= stopped by SIGTERM.



so you can inspect it and try the command by hand.

04 Check the file is where you think it is

Inside that shell: `pwd` , `ls -la` , `cat` the config. A wrong `WORKDIR` or a file excluded by `.dockerignore` explains most "not found" errors.

05 Run the command manually

Type the exact `CMD` yourself in that shell. The error message you get interactively is usually far more informative than the one in the logs.

06 Inspect the resolved configuration

`docker inspect NAME` shows the real endpoint, command, environment, mounts, and networks after every default and override has been applied. This settles "but I set that".

BASH

Copy

```
# 127 = command not found: is the binary there, and is PATH right?
docker run -it --entrypoint sh myapp:1.0
> which uvicorn
> ls -la /app
> echo "$PATH"

# 137 = killed, almost always the memory limit
docker inspect NAME --format '{{.State.ExitCode}} {{.State.OOMKilled}}'

# "but I set that environment variable"
docker inspect NAME --format '{{range .Config.Env}}{{println .}}{{end}}'
```

Run it in the foreground first

When something new does not work, drop the `-d` . Watching a container fail in your terminal, with its output arriving live, is almost always faster than starting it detached and then going to fetch the logs.



1. Produce a **127**: change `CMD` to a binary that does not exist, rebuild, run, and read the exit code.
2. Produce a **1**: make your app raise an exception on startup.
3. Produce a **137**: run with `-m 16m` so it is killed by the memory limit, then confirm with the `OOMKilled` format string.
4. For each one, get to the answer using the steps above rather than by remembering what you broke.

Expected result – three recognisable failures and three distinct exit codes. Having produced them deliberately, you will recognise each instantly when it happens for real, which is the difference between a five-minute fix and a lost afternoon.

Keeping your machine clean: prune and reclaim

Docker accumulates. Images you pulled once, containers you forgot to remove, build cache, and volumes from projects you stopped last spring, and then one day a build fails with `no space left on device`.

BASH

Copy

```
docker system df          # what is using space, by category
docker system df -v       # per-image, per-container, per-volume detail

docker ps -a              # stopped containers still hold their writable layer
docker container prune     # remove all stopped containers
docker image prune         # remove dangling (untagged) images
docker builder prune       # remove build cache
docker system prune        # containers, networks, dangling images, build cache
```

Start with `docker system df`. It tells you which category is large, so you can stop guessing. Usually it is either the build cache or images.



the database of a project you are not running right now. Run `docker volume ls` and look at the list before you type it. This is the one Docker command that can lose work you cannot get back.

Worth understanding: a **dangling** image is one with no tag, usually because you rebuilt the same tag and the old image lost its name. Those are safe to remove. `docker image prune -a` is more aggressive. It removes every image not used by a container, which means re-pulling next time.

► TRY IT

1. Run `docker system df` and note the reclaimable figure in each row.
2. Run `docker ps -a` and count how many stopped containers you have accumulated during this guide.
3. Run `docker container prune` and then `docker image prune`, and compare `docker system df` before and after.
4. Run `docker volume ls` and identify which volumes you would lose to a `--volumes prune`.

Expected result – usually more reclaimable space than expected, and a list of volumes that makes the warning above concrete. Getting into the habit of running `system df` occasionally is what stops "no space left on device" from ever surprising you.

Putting it all together

Everything above, in one project. Nothing here is new. Read it as a whole and you should recognise every line and be able to say why it is there.

```
.dockerignore
```

Copy



```
.gitignore
node_modules
__pycache__
*.pyc
.venv
.env
.env.*
dist
build
*.log
.pytest_cache
.DS_Store
```

Dockerfile

Copy

```
# Pinned and slim: reproducible, and a fraction of the full image's size
FROM python:3.11-slim

# Unbuffered stdout, so `docker logs` shows lines as they happen
ENV PYTHONUNBUFFERED=1 \
    LOG_LEVEL=info

# A real non-root user with a fixed numeric id
RUN useradd --create-home --uid 10001 appuser

WORKDIR /app

# Dependencies first – this layer stays cached while you edit source
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Source last, owned by the app's user
COPY --chown=appuser:appuser . .

# Drop privileges after everything that needed them
USER 10001

# Documentation only; `-p` or Compose `ports:` is what publishes
EXPOSE 8000

# Exec form, so the app is PID 1 and receives SIGTERM.
```



compose.yaml

Copy

```
services:
  api:
    build: .
    ports: ['8000:8000']
    environment:
      DATABASE_URL: postgres://postgres:secret@db:5432/app
      LOG_LEVEL: debug # override of the image's baked default
    depends_on: [db] # startup order (readiness comes at Mid level)
    restart: unless-stopped

  db:
    image: postgres:16 # pinned major version
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: app
    volumes: ['pgdata:/var/lib/postgresql/data'] # named volume: state survives
    restart: unless-stopped

volumes:
  pgdata:
```

BASH

Copy

```
docker compose up -d --build # build the image and start both services
docker compose logs -f api # watch it come up
docker compose exec api sh # get inside when something looks wrong
docker compose down # stop everything; the database volume survives
```

Ten decisions in there are the whole lesson of this page, and each maps to a section above:

Line	Why it is there
<code>.dockerignore</code> written first	Smaller context, and <code>.env</code> never reaches the image
<code>FROM python:3.11-slim</code>	Pinned, not <code>:latest</code> ; slim, not the full 1 GB base



<code>ENV PYTHONUNBUFFERED=1</code>	<code>docker logs</code> shows output immediately
<code>COPY requirements.txt</code> before source	The expensive install layer stays cached
<code>--no-cache-dir</code>	pip's wheel cache is dead weight in the image
<code>COPY --chown + USER 10001</code>	The process is not root
<code>EXPOSE 8000</code>	Documents the port; publishing is separate
Exec-form <code>CMD</code>	The app is PID 1, so <code>docker stop</code> is graceful
<code>--host 0.0.0.0</code>	The published port reaches the app
Named volume for Postgres	The data outlives the container

► TRY IT: THE ONE THAT MATTERS

1. Take this setup into a project you wrote, adapting the base image and commands to your language.
2. Get it running with `docker compose up -d --build`, then confirm from inside the api container that it can reach `db`.
3. Verify three things deliberately: `docker run --rm yourimage id` shows a non-root uid; a source edit rebuilds in seconds; and `docker compose down` followed by `up` keeps your database contents.
4. Write a three-line "Running locally" section in your README that is just the commands above.

Expected result – a containerised project on real code, with cached rebuilds, a non-root process, and data that survives. Build it once and the page turns into reference material, and the README section is what makes it useful to somebody other than you.

What you can now do, and what comes next



data in volumes, connect containers by name, run as a non-root user, start a multi-service stack with Compose, debug a container that refuses to start, and keep your disk under control. That is a working practitioner's toolkit, enough to containerise real projects and own the Dockerfiles in a repository.

Can you...	
Explain a container versus a VM?	Shared kernel, a process with a restricted view
Explain an image versus a container?	Read-only template versus running instance
Say why <code>docker run ubuntu</code> exits at once?	The main process finished
Say what <code>EXPOSE</code> does?	Documents: <code>-p</code> publishes
Order a Dockerfile for fast rebuilds?	Dependency manifest before source
Say why deleting a file in a later layer is not enough?	Layers are additive
Fix "the published port refuses connections"?	Listen on <code>0.0.0.0</code>
Keep a database's data?	A named volume
Make two containers talk?	A user-defined network, address by name
Get a shell in an image that crashes on start?	<code>--entrypoint sh</code>
Read a container's real configuration?	<code>docker inspect --format</code>
Start an app plus its database with one command?	<code>docker compose up -d</code>

Mid-level takes every one of those topics further: the layer cache's exact rules and how to make it work in CI, multi-stage builds that cut image size by an order of magnitude, network drivers and DNS in depth, volume backup and permission handling, Compose with health checks and per-environment override files, signals and graceful shutdown, registries and immutable tags, and debugging with `docker diff` , `docker events` , and a network sidecar.



bounded, keeping secrets out of layers with buildkit mounts, supply-chain controls including SBOMs, scanning, signing, and provenance, multi-architecture builds, resource limits and the OOM killer, GPU and machine-learning images, where Docker stops and an orchestrator begins, and debugging production while it is on fire.

NEXT UP

Beginner · Interview Review



Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

Roadmaps

Student Guides

Courses

The MLOps Practitioner

Sessions

Articles



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**